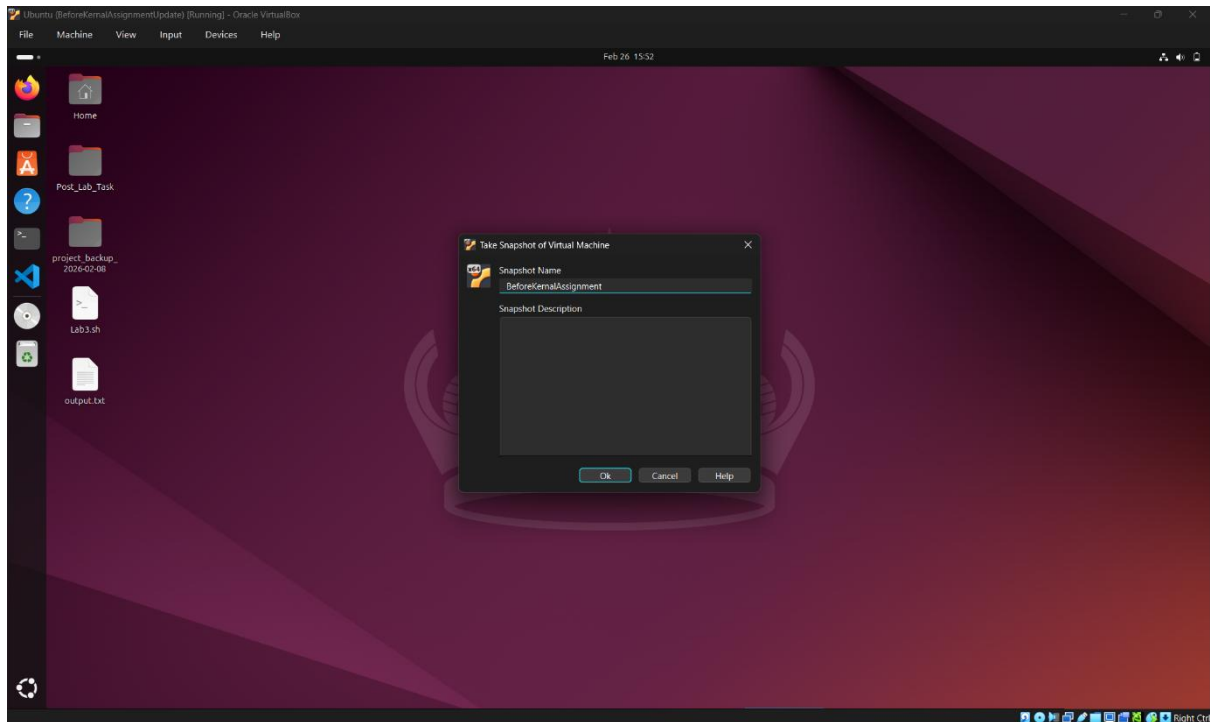


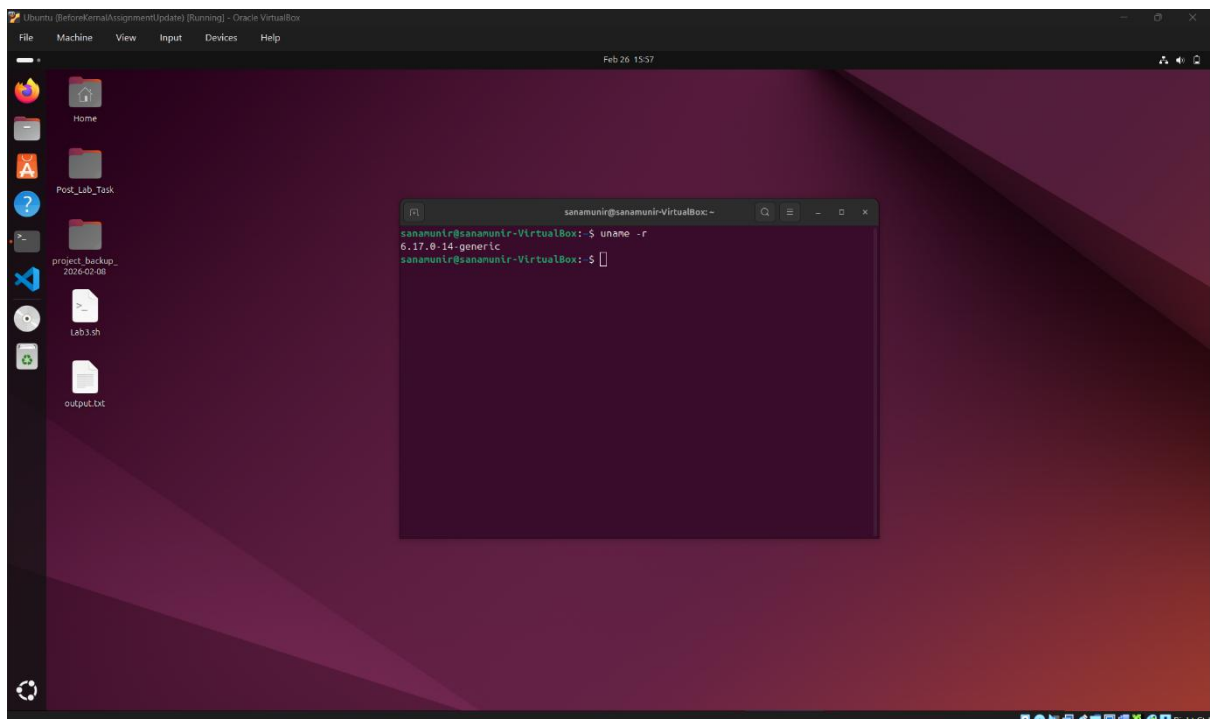
Step 1: TAKE SNAPSHOT

A snapshot of the virtual machine was taken before kernel compilation. This ensures the system can be restored if kernel installation fails.



Step 2: CHECK CURRENT KERNAL VERSION

The current running kernel version was checked using `uname -r`. The system is running kernel 6.17.0-14-generic.



Step 3: INSTALL REQUIRED DEPENDENCIES

Required development libraries and build tools were installed. These packages are necessary for compiling and configuring the Linux kernel.

```
Ubuntu (BeforeKernelAssignmentUpdate) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

sanamunir@sanamunir-VirtualBox:~$ uname -r
6.17.0-14-generic
sanamunir@sanamunir-VirtualBox:~$ sudo apt update
[sudo] password for sanamunir:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:2 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Hit:3 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Get:4 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 https://packages.microsoft.com/repos/code stable/main amd64 Packages [22.8 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,491 kB]
Get:8 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,800 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [239 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [9,932 B]
Get:12 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2,557 kB]
Get:13 http://pk.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [331 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [594 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:16 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [937 kB]
Get:17 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:18 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.5 kB]
Get:19 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2,735 kB]
Get:20 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [217 kB]
Get:21 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:22 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.1 kB]
Get:23 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6,732 B]
Get:24 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:25 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [630 kB]
Get:26 http://pk.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:27 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,558 kB]
Get:28 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [316 kB]
Get:29 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:30 http://pk.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [32.7 kB]
Get:31 http://pk.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [7,044 B]
Get:32 http://pk.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:33 http://pk.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,284 B]
Get:34 http://pk.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:35 http://pk.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:36 http://pk.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 14.6 MB in 24s (602 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
68 packages can be upgraded. Run 'apt list --upgradable' to see them.
sanamunir@sanamunir-VirtualBox:~$
```

```
sanamunir@sanamunir-VirtualBox:~$ sudo apt install -y build-essential libncurses-dev bison flex bc libssl-dev libelf-dev dwarves libdw-dev dpkg-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.10ubuntu1).
libncurses-dev is already the newest version (6.4+20240113-1ubuntu2).
bison is already the newest version (2:3.8.2+dfsg-1build2).
flex is already the newest version (2.6.4-8.2build1).
bc is already the newest version (1.07.1-3ubuntu4).
libssl-dev is already the newest version (3.0.13-0ubuntu3.7).
libelf-dev is already the newest version (0.190-1.1ubuntu0.1).
dpkg-dev is already the newest version (1.22.6ubuntu6.5).
dpkg-dev set to manually installed.
The following additional packages will be installed:
  liblzma-dev pahole
Suggested packages:
  liblzma-doc
The following NEW packages will be installed:
  dwarves libdw-dev liblzma-dev pahole
0 upgraded, 4 newly installed, 0 to remove and 3 not upgraded.
Need to get 795 kB of archives.
After this operation, 4,254 kB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 liblzma-dev amd64 5.6.1+really5.4.5-1ubuntu0.2 [176 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 libdw-dev amd64 0.190-1.1ubuntu0.1 [340 kB]
Get:3 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 pahole amd64 1.25-0ubuntu3 [277 kB]
Get:4 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 dwarves all 1.25-0ubuntu3 [2,186 B]
Fetched 795 kB in 12s (69.0 kB/s)
Selecting previously unselected package liblzma-dev:amd64.
(Reading database ... 199085 files and directories currently installed.)
Preparing to unpack .../liblzma-dev_5.6.1+really5.4.5-1ubuntu0.2_amd64.deb ...
Unpacking liblzma-dev:amd64 (5.6.1+really5.4.5-1ubuntu0.2) ...
Selecting previously unselected package libdw-dev:amd64.
Preparing to unpack .../libdw-dev_0.190-1.1ubuntu0.1_amd64.deb ...
Unpacking libdw-dev:amd64 (0.190-1.1ubuntu0.1) ...
Selecting previously unselected package pahole.
Preparing to unpack .../pahole_1.25-0ubuntu3_amd64.deb ...
Unpacking pahole (1.25-0ubuntu3) ...
Selecting previously unselected package dwarves.
Preparing to unpack .../dwarves_1.25-0ubuntu3_all.deb ...
Unpacking dwarves (1.25-0ubuntu3) ...
Setting up pahole (1.25-0ubuntu3) ...
Setting up liblzma-dev:amd64 (5.6.1+really5.4.5-1ubuntu0.2) ...
Setting up libdw-dev:amd64 (0.190-1.1ubuntu0.1) ...
Setting up dwarves (1.25-0ubuntu3) ...
Processing triggers for man-db (2.12.0-4build2) ...
sanamunir@sanamunir-VirtualBox:~$
```

Step 4: CREATE KERNEL DIRECTORY

A dedicated directory was created to store kernel source files to maintain system organization.

```
sanamunir@sanamunir-VirtualBox:~$ mkdir -p ~/kernel
sanamunir@sanamunir-VirtualBox:~$ cd ~/kernel
```

Step 5: DOWNLOAD KERNEL 6.17

The Linux kernel version 6.17 (same as our current generic one for system compatibility) was downloaded from the official Linux Kernel Archives using wget.

```
sanamunir@sanamunir-VirtualBox:~/kernel$ wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.17.tar.xz
--2026-02-26 19:50:17-- https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.17.tar.xz
Resolving cdn.kernel.org (cdn.kernel.org)... 151.101.1.176, 151.101.65.176, 151.101.129.176, ...
Connecting to cdn.kernel.org (cdn.kernel.org)|151.101.1.176|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 153382068 (146M) [application/x-xz]
Saving to: 'linux-6.17.tar.xz'

linux-6.17.tar.xz                               100%[=====]
2026-02-26 19:51:56 (1.49 MB/s) - 'linux-6.17.tar.xz' saved [153382068/153382068]

sanamunir@sanamunir-VirtualBox:~/kernel$
```

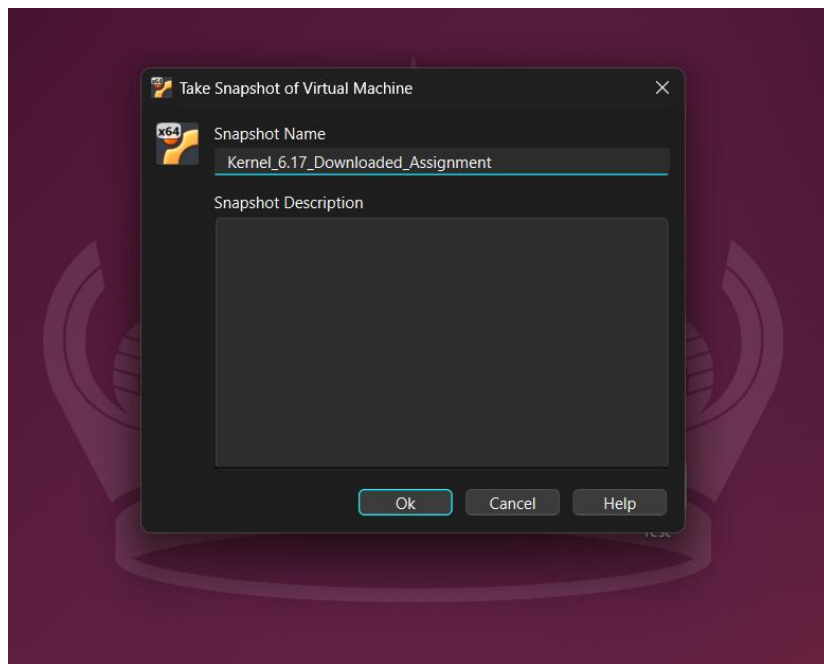
```
=====] 146.28M  1.42MB/s   in 98s
```

Step 6: EXTRACT KERNEL

The compressed kernel source archive was extracted. The extracted directory contains all kernel source code including drivers, memory management, networking, and core system files

```
sanamunir@sanamunir-VirtualBox:~/kernel$ tar -xf linux-6.17.tar.xz
sanamunir@sanamunir-VirtualBox:~/kernel$ cd linux-6.17
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ ls
arch  certs  CREDITS  Documentation  fs      init      ipc      Kconfig  lib
MAINTAINERS  mm      README  samples  security  tools  virt
block  COPYING  crypto  drivers  include  io_uring  Kbuild  kernel  LICENSES
Makefile  net  rust  scripts  sound  usr
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 7: TAKE SNAPSHOT



Step 8: COPY CURRENT KERNEL CONFIGURATION

The configuration file of the currently running kernel (6.17.0-14-generic) was copied into the new kernel source directory. This ensures the new kernel will use compatible settings for hardware and modules.

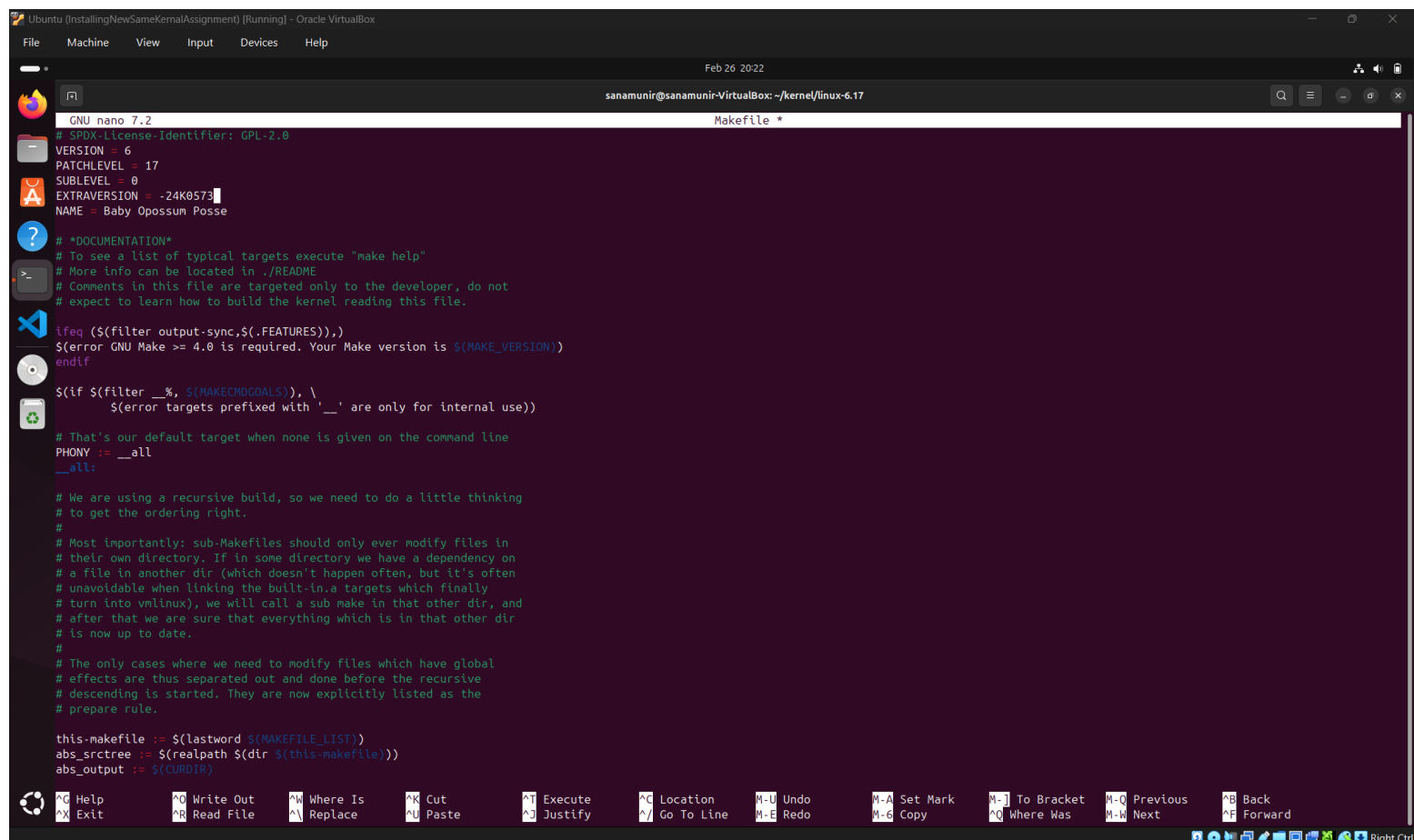
The command `make olddefconfig` was then run, which reads the copied configuration and fills in any missing options with default values. This step prepares the kernel for compilation

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ cp /boot/config-6.17.0-14-generic .config
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ make olddefconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
.config:11452:warning: symbol value 'm' invalid for ANDROID_BINDER_IPC
.config:11453:warning: symbol value 'm' invalid for ANDROID_BINDERFS
#
# configuration written to .config
#
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 9: MODIFY KERNEL NAME

The EXTRAVERSION field in the Makefile was modified to uniquely identify the newly compiled kernel as 6.17.0-24K0573.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ sudo nano Makefile
[sudo] password for sanamunir:
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```



```
GNU nano 7.2
# SPDX-License-Identifier: GPL-2.0
VERSION = 6
PATCHLEVEL = 17
SUBLEVEL = 0
EXTRAVERSION = -24K0573
NAME = Baby Opossum Posse

# *DOCUMENTATION*
# To see a list of typical targets execute "make help"
# More info can be located in ./README
# Comments in this file are targeted only to the developer, do not
# expect to learn how to build the kernel reading this file.

ifeq ($(filter output-sync,$(.FEATURES)),)
$(error GNU Make >= 4.0 is required. Your Make version is $(MAKE_VERSION))
endif

$(if $(filter __%, $(MAKECMDGOALS)), \
    $(error targets prefixed with '__' are only for internal use))

# That's our default target when none is given on the command line
PHONY := __all
__all:

# We are using a recursive build, so we need to do a little thinking
# to get the ordering right.
#
# Most importantly: sub-Makefiles should only ever modify files in
# their own directory. If in some directory we have a dependency on
# a file in another dir (which doesn't happen often, but it's often
# unavoidable when linking the built-in.a targets which finally
# turn into vmlinux), we will call a sub make in that other dir, and
# after that we are sure that everything which is in that other dir
# is now up to date.
#
# The only cases where we need to modify files which have global
# effects are thus separated out and done before the recursive
# descending is started. They are now explicitly listed as the
# prepare rule.

this-makefile := $(lastword $(MAKEFILE_LIST))
abs_srctree := $(realpath $(dir $(this-makefile)))
abs_output := $(CURDIR)

^G Help
^X Exit
^O Write Out
^R Read File
^W Where Is
^N Replace
^K Cut
^U Paste
^T Execute
^J Justify
^C Location
^_ Go To Line
^M-U Undo
^M-E Redo
^M-A Set Mark
^M-G Copy
^M-J To Bracket
^M-Q Where Was
^M-Q Previous
^M-W Next
^B Back
^F Forward
```

Step 10: **CREATING HELLO.C IN KERNEL SOURCE**

A new source file named hello.c was created inside the kernel/ directory of the Linux kernel source tree.

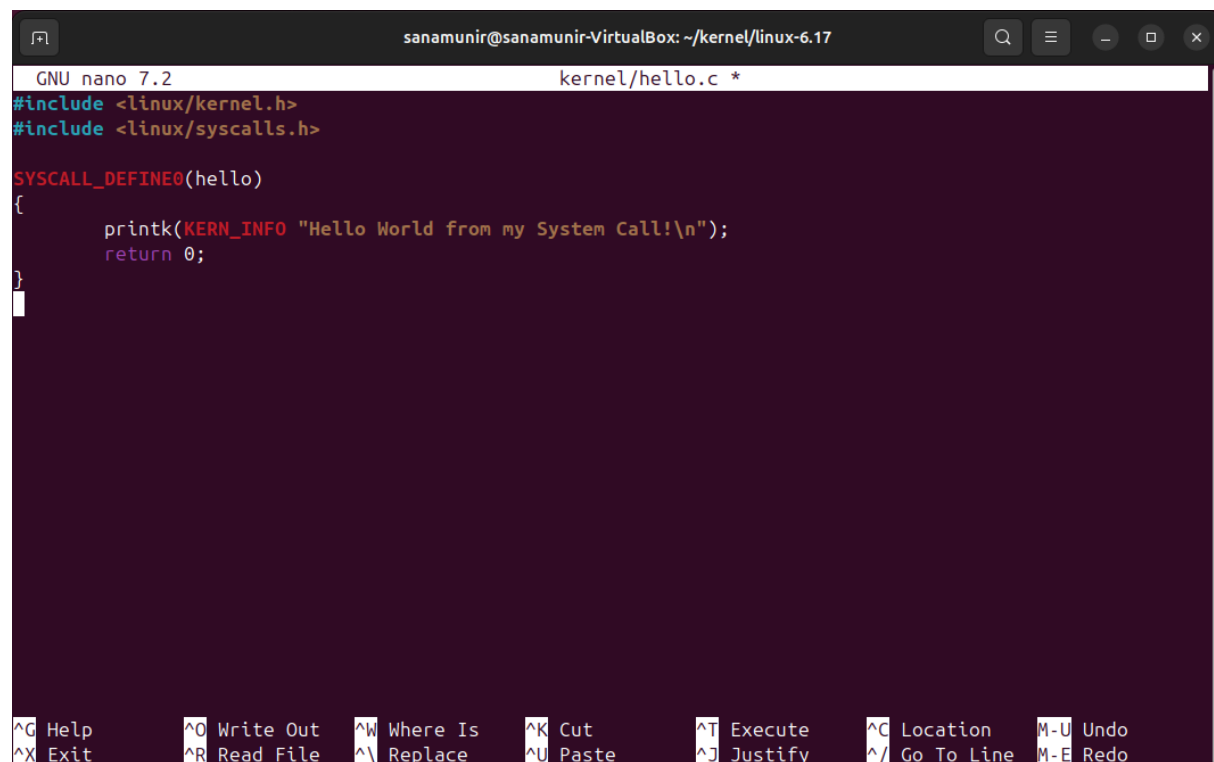
This file defines a custom system call using the SYSCALL_DEFINE0 macro. The macro creates a system call named hello that takes zero arguments.

Inside the system call, the printk() function is used to print the message:

"Hello World from my System Call!"

to the kernel log buffer. The function returns 0 to indicate successful execution.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ sudo nano kernel/hello.c
[sudo] password for sanamunir:
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```



```
GNU nano 7.2 kernel/hello.c *
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(hello)
{
    printk(KERN_INFO "Hello World from my System Call!\n");
    return 0;
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

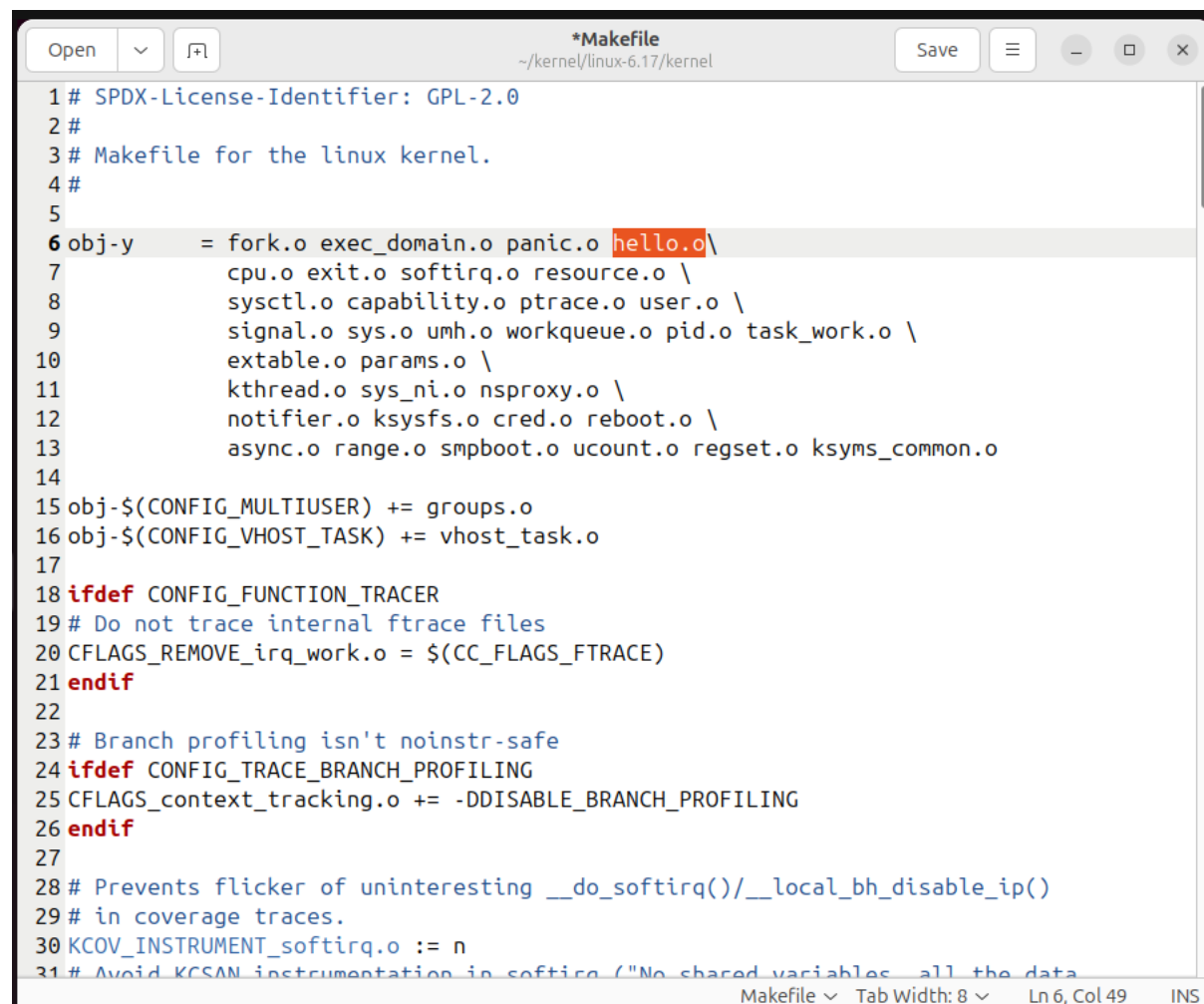
Step 11: MODIFYING KERNEL/MAKEFILE

The kernel/Makefile was modified to include the newly created hello.c file in the kernel build process.

The line **hello.o** was added to the obj-y list.

The **obj-y** variable specifies object files that must be compiled and linked into the main kernel image. By adding hello.o, the new system call implementation becomes part of the kernel during compilation.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ gedit kernel/Makefile
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```



```
1 # SPDX-License-Identifier: GPL-2.0
2 #
3 # Makefile for the linux kernel.
4 #
5
6 obj-y      = fork.o exec_domain.o panic.o hello.o \
7            cpu.o exit.o softirq.o resource.o \
8            sysctl.o capability.o ptrace.o user.o \
9            signal.o sys.o umh.o workqueue.o pid.o task_work.o \
10           extable.o params.o \
11           kthread.o sys_ni.o nsproxy.o \
12           notifier.o ksysfs.o cred.o reboot.o \
13           async.o range.o smpboot.o ucount.o regset.o ksyms_common.o
14
15 obj-$(CONFIG_MULTIUSER) += groups.o
16 obj-$(CONFIG_VHOST_TASK) += vhost_task.o
17
18 ifdef CONFIG_FUNCTION_TRACER
19 # Do not trace internal ftrace files
20 CFLAGS_REMOVE_irq_work.o = $(CC_FLAGS_FTRACE)
21 endif
22
23 # Branch profiling isn't noinstr-safe
24 ifdef CONFIG_TRACE_BRANCH_PROFILING
25 CFLAGS_context_tracking.o += -DDISABLE_BRANCH_PROFILING
26 endif
27
28 # Prevents flicker of uninteresting __do_softirq()/__local_bh_disable_ip()
29 # in coverage traces.
30 KCOV_INSTRUMENT_softirq.o := n
31 # Avoid KCSAN instrumentation in softirq ("No shared variables, all the data
```


Step 12: DECLARING THE SYSTEM CALL PROTOTYPE

The header file `include/linux/syscalls.h` was modified to declare the prototype of the newly created system call.

The following line was added before the `#endif` statement at the end of the file:

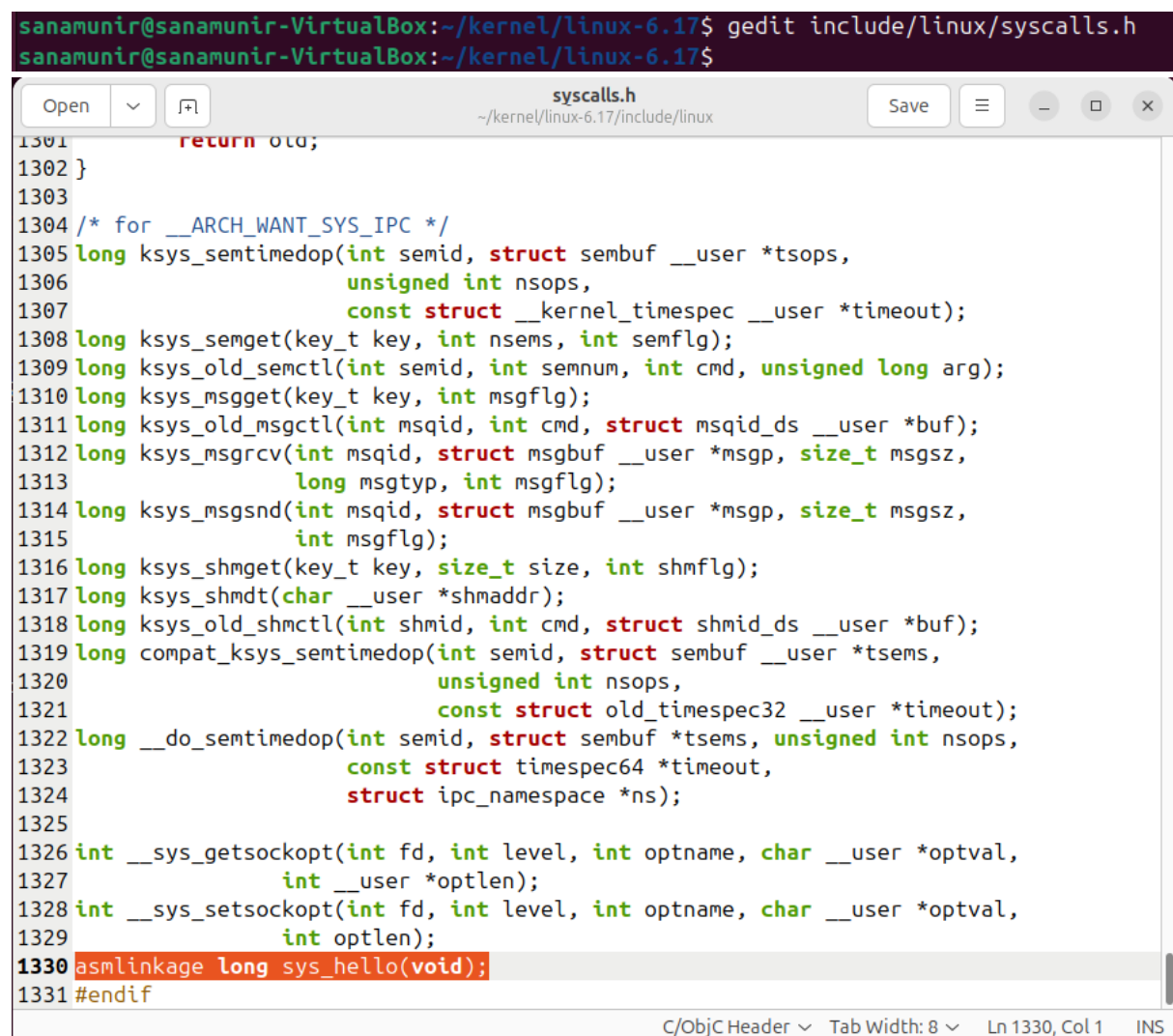
asmlinkage long sys_hello(void);

This declaration informs the kernel about the existence of the `sys_hello` system call function.

The ***asmlinkage*** keyword ensures that the system call arguments are retrieved from the CPU registers according to the system call calling convention.

The function returns a long value and takes no arguments, matching the definition provided using `SYSCALL_DEFINE0(hello)`.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ gedit include/linux/syscalls.h
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```



```
1301     return 0;
1302 }
1303
1304 /* for __ARCH_WANT_SYS_IPC */
1305 long ksys_semtimedop(int semid, struct sembuf __user *tsops,
1306                     unsigned int nsops,
1307                     const struct __kernel_timespec __user *timeout);
1308 long ksys_semget(key_t key, int nsems, int semflg);
1309 long ksys_old_semctl(int semid, int semnum, int cmd, unsigned long arg);
1310 long ksys_msgget(key_t key, int msgflg);
1311 long ksys_old_msgctl(int msqid, int cmd, struct msqid_ds __user *buf);
1312 long ksys_msgrcv(int msqid, struct msgbuf __user *msgp, size_t msgsz,
1313                 long msgtyp, int msgflg);
1314 long ksys_msgsnd(int msqid, struct msgbuf __user *msgp, size_t msgsz,
1315                 int msgflg);
1316 long ksys_shmget(key_t key, size_t size, int shmflg);
1317 long ksys_shmdt(char __user *shmaddr);
1318 long ksys_old_shmctl(int shmid, int cmd, struct shmid_ds __user *buf);
1319 long compat_ksys_semtimedop(int semid, struct sembuf __user *tsems,
1320                             unsigned int nsops,
1321                             const struct old_timespec32 __user *timeout);
1322 long __do_semtimedop(int semid, struct sembuf *tsems, unsigned int nsops,
1323                     const struct timespec64 *timeout,
1324                     struct ipc_namespace *ns);
1325
1326 int __sys_getsockopt(int fd, int level, int optname, char __user *optval,
1327                     int __user *optlen);
1328 int __sys_setsockopt(int fd, int level, int optname, char __user *optval,
1329                     int optlen);
1330 asmlinkage long sys_hello(void);
1331 #endif
```

C/ObjC Header ▾ Tab Width: 8 ▾ Ln 1330, Col 1 INS

Step 13: ADDING ENTRY IN SYSTEM CALL TABLE

The system call table for the x86_64 architecture was modified to register the newly created system call.

The file arch/x86/entry/syscalls/syscall_64.tbl contains the mapping between system call numbers and their corresponding kernel functions.

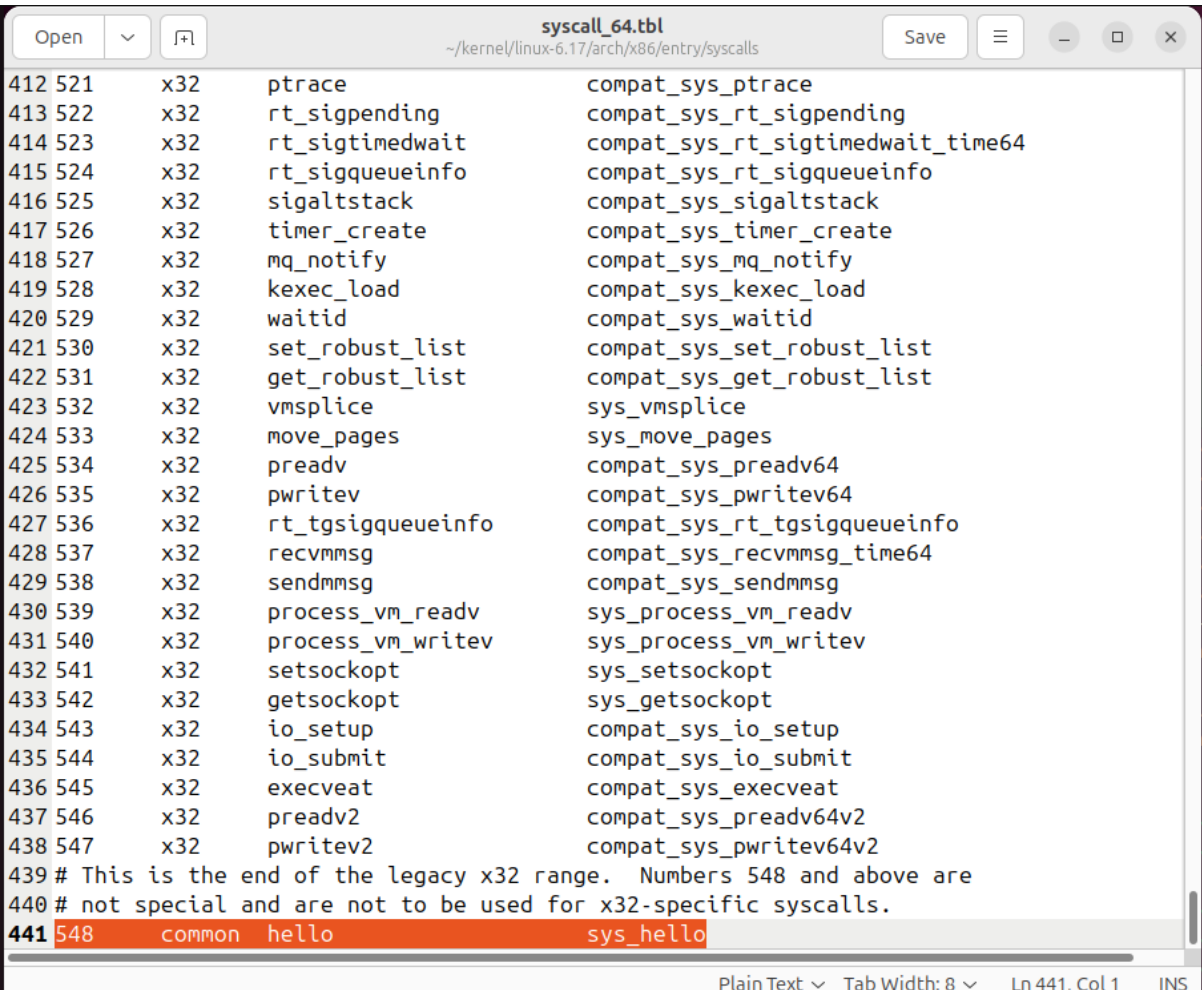
The following line was added at the end of the file: **548 common hello sys_hello**

Here:

- **548** is the system call number (chosen as an unused number).
- **common** indicates that the system call is valid for both 32-bit and 64-bit modes (where applicable).
- **hello** is the name of the system call.
- **sys_hello** is the kernel function that implements the system call.

This step officially registers the new system call within the Linux kernel so that it can be invoked from user space.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ gedit arch/x86/entry/syscalls/syscall_64.tbl
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```



```
412 521    x32    ptrace          compat_sys_ptrace
413 522    x32    rt_sigpending  compat_sys_rt_sigpending
414 523    x32    rt_sigtimedwait compat_sys_rt_sigtimedwait_time64
415 524    x32    rt_sigqueueinfo compat_sys_rt_sigqueueinfo
416 525    x32    sigaltstack   compat_sys_sigaltstack
417 526    x32    timer_create   compat_sys_timer_create
418 527    x32    mq_notify      compat_sys_mq_notify
419 528    x32    kexec_load     compat_sys_kexec_load
420 529    x32    waitid         compat_sys_waitid
421 530    x32    set_robust_list compat_sys_set_robust_list
422 531    x32    get_robust_list compat_sys_get_robust_list
423 532    x32    vmsplice       sys_vmsplice
424 533    x32    move_pages     sys_move_pages
425 534    x32    preadv         compat_sys_preadv64
426 535    x32    pwritev        compat_sys_pwritev64
427 536    x32    rt_tgsigqueueinfo compat_sys_rt_tgsigqueueinfo
428 537    x32    recvmmsg       compat_sys_recvmmsg_time64
429 538    x32    sendmmsg       compat_sys_sendmmsg
430 539    x32    process_vm_readv sys_process_vm_readv
431 540    x32    process_vm_writev sys_process_vm_writev
432 541    x32    setsockopt     sys_setsockopt
433 542    x32    getsockopt     sys_getsockopt
434 543    x32    io_setup       compat_sys_io_setup
435 544    x32    io_submit      compat_sys_io_submit
436 545    x32    execveat       compat_sys_execveat
437 546    x32    preadv2        compat_sys_preadv64v2
438 547    x32    pwritev2       compat_sys_pwritev64v2
439 # This is the end of the legacy x32 range. Numbers 548 and above are
440 # not special and are not to be used for x32-specific syscalls.
441 548      common  hello        sys_hello
```

Plain Text ▾ Tab Width: 8 ▾ Ln 441, Col 1 INS

Step 14: INSTALLING TEXT-PROCESSING UTILITY

gawk (GNU Awk) is a text-processing utility required by the Linux kernel build system. It is used by various build scripts during kernel compilation to process configuration and source files. Installing it ensures that the kernel compilation process completes without missing dependency errors.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ sudo apt install gawk -y
[sudo] password for sanamunir:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libsigsegv2
Suggested packages:
  gawk-doc
The following NEW packages will be installed:
  gawk libsigsegv2
0 upgraded, 2 newly installed, 0 to remove and 3 not upgraded.
Need to get 478 kB of archives.
After this operation, 1,826 kB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libsigsegv2 amd64 2.14-1ubuntu2 [15.0 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 gawk amd64 1:5.2.1-2build3 [463 kB]
Fetched 478 kB in 2s (248 kB/s)
Selecting previously unselected package libsigsegv2:amd64.
(Reading database ... 201048 files and directories currently installed.)
Preparing to unpack .../libsigsegv2_2.14-1ubuntu2_amd64.deb ...
Unpacking libsigsegv2:amd64 (2.14-1ubuntu2) ...
Setting up libsigsegv2:amd64 (2.14-1ubuntu2) ...
Selecting previously unselected package gawk.
(Reading database ... 201055 files and directories currently installed.)
Preparing to unpack .../gawk_1%3a5.2.1-2build3_amd64.deb ...
Unpacking gawk (1:5.2.1-2build3) ...
Setting up gawk (1:5.2.1-2build3) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 15: KERNEL CONFIGURATION WITH make menuconfig

The command `make menuconfig` was used to open the Linux kernel **configuration menu**.

This menu allows users to customize kernel options such as:

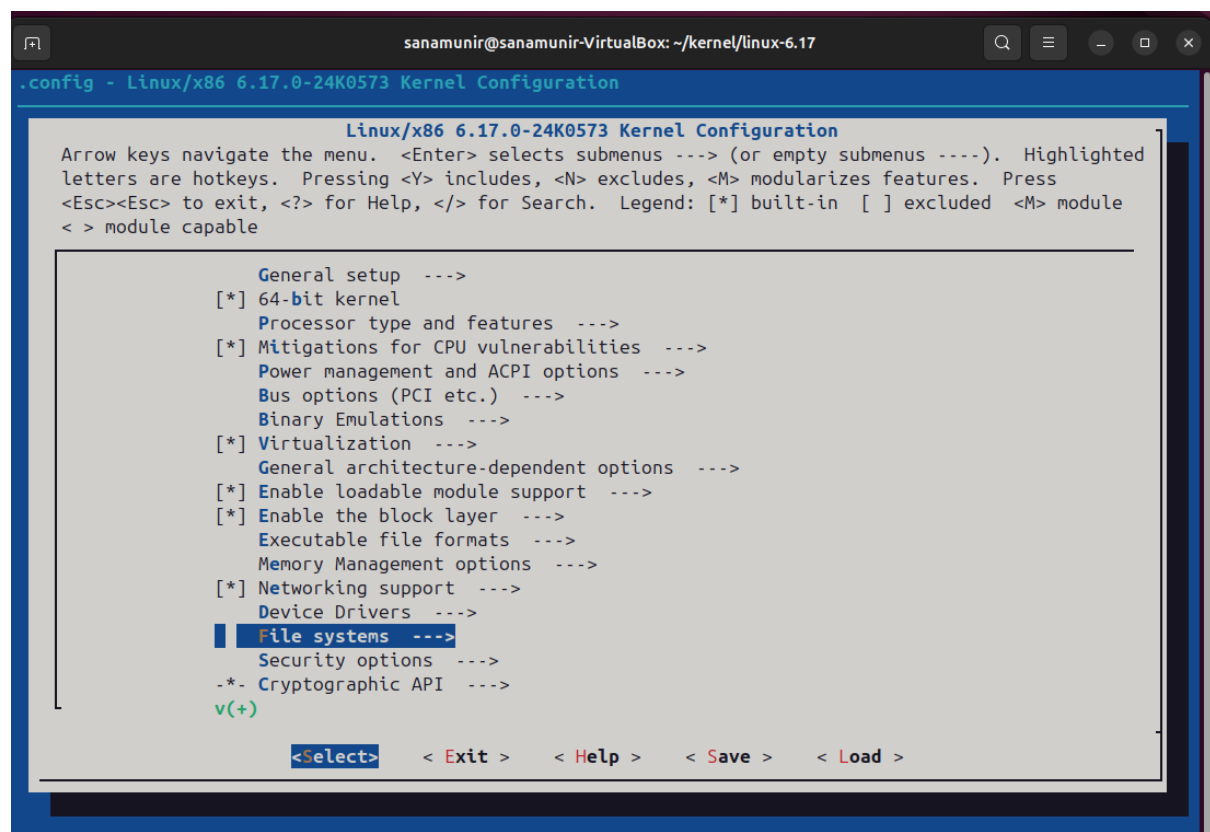
- File systems / Device drivers / Network support / Security options

In this step, the configuration was reviewed, and the default settings were accepted. The configuration was then **saved** as `.config` in the kernel source directory.

This `.config` file contains all the settings that the kernel build system will use to compile the kernel.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ make menuconfig
HOSTCC  scripts/kconfig/mconf.o
HOSTCC  scripts/kconfig/lxdialog/checklist.o
HOSTCC  scripts/kconfig/lxdialog/inputbox.o
HOSTCC  scripts/kconfig/lxdialog/menubox.o
HOSTCC  scripts/kconfig/lxdialog/textbox.o
HOSTCC  scripts/kconfig/lxdialog/util.o
HOSTCC  scripts/kconfig/lxdialog/yesno.o
HOSTCC  scripts/kconfig/mnconf-common.o
HOSTLD  scripts/kconfig/mconf

*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.
```



```
sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17
.config - Linux/x86 6.17.0-24K0573 Kernel Configuration
> File systems

File systems
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module
< > module capable

[*] Validate filesystem parameter description
< > Second extended fs support (DEPRECATED)
< > The Extended 3 (ext3) filesystem
[*] The Extended 4 (ext4) filesystem
[*] Use ext4 for ext2 file systems
[*] Ext4 POSIX Access Control Lists
[*] Ext4 Security Labels
[ ] Ext4 debugging support
[ ] JBD2 (ext4) debugging support
<M> JFS filesystem support
[*] JFS POSIX Access Control Lists
[*] JFS Security Labels
[ ] JFS debugging
[*] JFS statistics
<M> XFS filesystem support
[*] Support deprecated V4 (crc=0) format
[*] Support deprecated case-insensitive ascii (ascii-ci=1) format
[*] XFS Quota support
v(+)

<select> < Exit > < Help > < Save > < Load >
```

```
sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17
.config - Linux/x86 6.17.0-24K0573 Kernel Configuration

Linux/x86 6.17.0-24K0573 Kernel Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module
< > module capable

General setup --->
[*] 64-bit kernel
Processor type and features --->
[*] Mitigations for CPU vulnerabilities --->
Power management and ACPI options --->
Bus options (PCI etc.) --->
Binary Emulations --->
[*] Virtualization --->
General architecture-dependent options --->
[*] Enable loadable module support --->
[*] Enable the block layer --->
Executable file formats --->
Memory Management options --->
[*] Networking support --->
[*] Device Drivers --->
File systems --->
Security options --->
*- Cryptographic API --->
v(+)

<select> < Exit > < Help > < Save > < Load >
```

```
sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17
.config - Linux/x86 6.17.0-24K0573 Kernel Configuration
> Device Drivers

Device Drivers
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module
< > module capable
^(-)
  Bus devices --->
  Cache Drivers ----
  [*] Connector - unified userspace <-> kernelspace linker --->
  Firmware Drivers --->
  {M} fwctl device firmware access framework --->
  <M> GNSS receiver support --->
  <M> Memory Technology Device (MTD) support --->
  [ ] Device Tree and Open Firmware support ----
  <M> Parallel port support --->
  -* Plug and Play support --->
  [*] Block devices --->
      NVME Support --->
      Misc devices --->
      SCSI device support --->
  <*> Serial ATA and Parallel ATA drivers (libata) --->
      [*] Multiple devices driver support (RAID and LVM) --->
      <M> Generic Target Core Mod (TCM) and ConfigFS Infrastructure --->
      [*] Fusion MPT device support --->
  v(+)

  <Select>  <Exit>  <Help>  <Save>  <Load>
```

```
sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17
.config - Linux/x86 6.17.0-24K0573 Kernel Configuration
> Device Drivers > Serial ATA and Parallel ATA drivers (libata)

Serial ATA and Parallel ATA drivers (libata)
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module
< > module capable

--- Serial ATA and Parallel ATA drivers (libata)
[*] Verbose ATA error reporting
[*] "libata.force=" kernel parameter support
[*] ATA ACPI Support
[*] SATA Zero Power Optical Disc Drive (ZPODD) support
[*] SATA Port Multiplier support
*** Controllers with non-SFF native interface ***
<M> AHCI SATA support
(3) Default SATA Link Power Management policy
<M> Platform AHCI SATA support
<M> Synopsys DWC AHCI SATA support
<M> Initio 162x SATA support (Very Experimental)
<M> ACard AHCI variant (ATP 8620)
<M> Silicon Image 3124/3132 SATA support
[*] ATA SFF support (for legacy IDE and PATA)
    *** SFF controllers with custom DMA interface ***
<M> Pacific Digital ADMA support
<M> Pacific Digital SATA QStor support
v(+)

  <Select>  <Exit>  <Help>  <Save>  <Load>
```

```
sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17
.config - Linux/x86 6.17.0-24K0573 Kernel Configuration
> Device Drivers > NVME Support

NVME Support
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module
< > module capable

<M> NVMe Express block device
[*] NVMe multipath support
[ ] NVMe verbose error reporting
[*] NVMe hardware monitoring
<M> NVMe Express over Fabrics RDMA host driver
<M> NVMe Express over Fabrics FC host driver
<M> NVMe Express over Fabrics TCP host driver
[*] NVMe over Fabrics TCP TLS encryption support
[*] NVMe over Fabrics In-Band Authentication in host side
<M> NVMe Target support
[ ] NVMe Target debugfs support
[*] NVMe Target Passthrough support
<M> NVMe loopback device support
<M> NVMe over Fabrics RDMA target support
<M> NVMe over Fabrics FC target driver
< > NVMe over Fabrics FC Transport Loopback Test driver
<M> NVMe over Fabrics TCP target support
[*] NVMe over Fabrics TCP target TLS encryption support
v(+)

<Select> < Exit > < Help > < Save > < Load >
```

```
sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17
.config - Linux/x86 6.17.0-24K0573 Kernel Configuration
> Device Drivers

Device Drivers
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted
letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press
<Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ] excluded <M> module
< > module capable

[*] PCI support --->
<M> PCCard (PCMCIA/CardBus) support --->
<*> RapidIO support --->
[*] PC/104 support
Generic Driver Options --->
Bus devices ----
Cache Drivers ----
[*] Connector - unified userspace <-> kernelspace linker --->
Firmware Drivers --->
{M} fwctl device firmware access framework --->
<M> GNSS receiver support --->
<M> Memory Technology Device (MTD) support --->
[ ] Device Tree and Open Firmware support ----
<M> Parallel port support --->
-* Plug and Play support --->
[*] Block devices --->
NVMe Support --->
Misc devices --->
v(+)

<Select> < Exit > < Help > < Save > < Load >
```

```
Enter a filename to which this configuration
should be saved as an alternate. Leave blank to
abort.

.config
< Ok > < Help >
```


Step 16: **KERNEL COMPILATION WITH make -j\$(nproc)** [Took 3 hrs]

The command `make -j$(nproc)` compiles the Linux kernel using all available CPU cores.

- `make` → invokes the kernel build system
- `-j$(nproc)` → automatically uses the number of CPU cores available in the system to speed up compilation (in this case 3)

During compilation, the build system compiles all kernel source files, builds object files (*.o), and generates the **kernel image** (bzImage) and **modules**.

RAN THIS COMMAND ACTUALLY as simple `make -j$(nproc)` gave certificate error so did this.

Set the certificate paths to empty during build

`make -j$(nproc) \`

`CONFIG_SYSTEM_TRUSTED_KEYS="" \`

`CONFIG_SYSTEM_REVOCATION_KEYS="" \`

`MODULE_SIG_KEY=""`

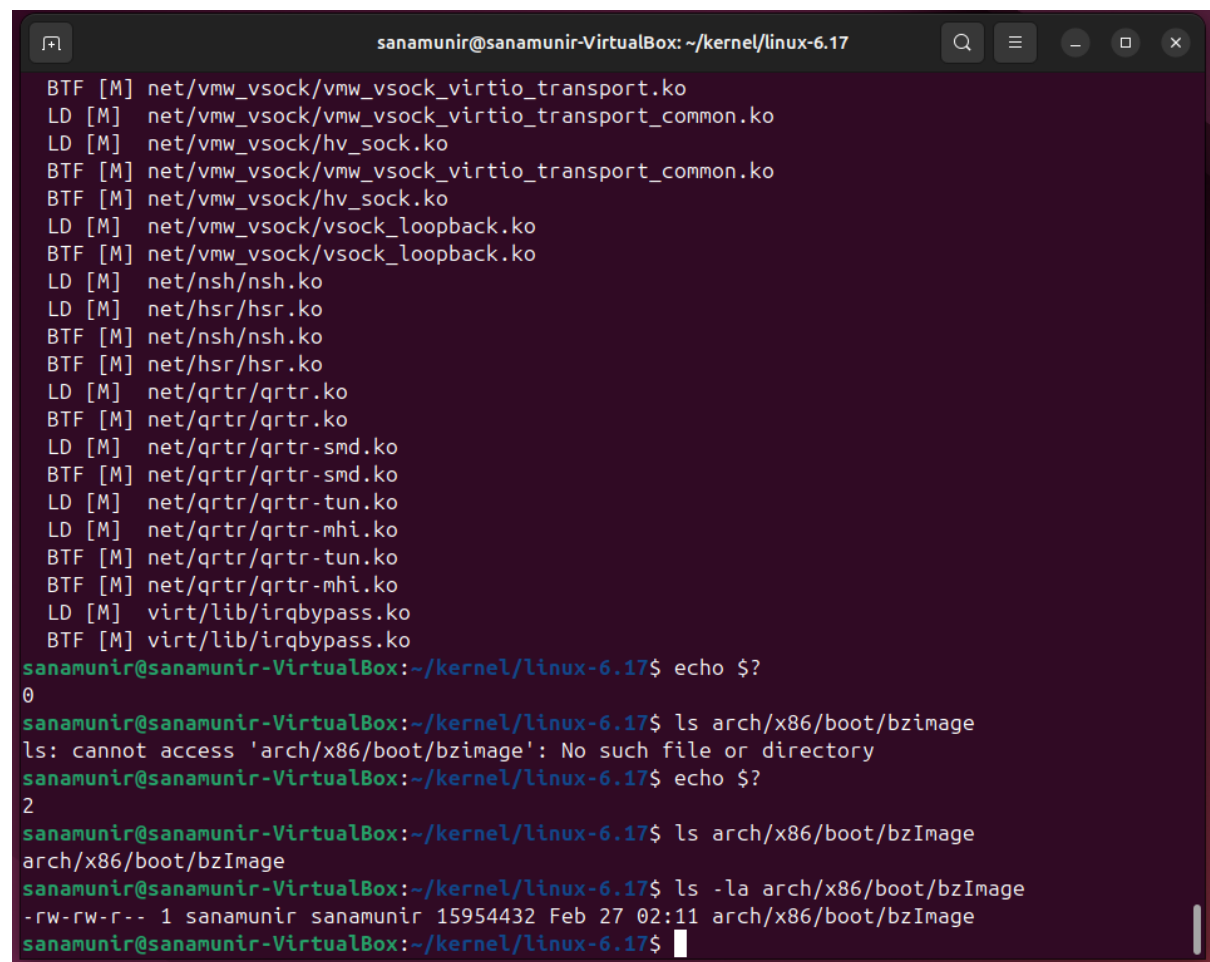
```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ make -j$(nproc)
SYNC      include/config/auto.conf.cmd
WRAP      arch/x86/include/generated/uapi/asm/bpf_perf_event.h
WRAP      arch/x86/include/generated/uapi/asm/errno.h
WRAP      arch/x86/include/generated/uapi/asm/fcntl.h
WRAP      arch/x86/include/generated/uapi/asm/ioctl.h
WRAP      arch/x86/include/generated/uapi/asm/ioctls.h
WRAP      arch/x86/include/generated/uapi/asm/ipcbuf.h
WRAP      arch/x86/include/generated/uapi/asm/param.h
SYSHDR    arch/x86/include/generated/uapi/asm/unistd_32.h
WRAP      arch/x86/include/generated/uapi/asm/poll.h
WRAP      arch/x86/include/generated/uapi/asm/resource.h
WRAP      arch/x86/include/generated/uapi/asm/socket.h
WRAP      arch/x86/include/generated/uapi/asm/sockios.h
WRAP      arch/x86/include/generated/uapi/asm/termbits.h
WRAP      arch/x86/include/generated/uapi/asm/termios.h
SYSHDR    arch/x86/include/generated/uapi/asm/unistd_64.h
WRAP      arch/x86/include/generated/uapi/asm/types.h
UPD       arch/x86/include/generated/asm/cpufeaturemasks.h
HOSTCC    arch/x86/tools/relocs_32.o
SYSHDR    arch/x86/include/generated/uapi/asm/unistd_x32.h
SYSTBL    arch/x86/include/generated/asm/syscalls_32.h
UPD       include/config/kernel.release
HOSTCC    arch/x86/tools/relocs_64.o
SYSHDR    arch/x86/include/generated/asm/unistd_32_ia32.h
SYSHDR    arch/x86/include/generated/asm/unistd_64_x32.h
SYSTBL    arch/x86/include/generated/asm/syscalls_64.h
HYPERCALLS arch/x86/include/generated/asm/xen-hypercalls.h
HOSTCC    arch/x86/tools/relocs_common.o
WRAP      arch/x86/include/generated/asm/early_ioremap.h
WRAP      arch/x86/include/generated/asm/fprobe.h
WRAP      arch/x86/include/generated/asm/mcs_spinlock.h
```


Step 17: VERIFYING SUCCESSFUL COMPILATION

After the kernel build process completed, the command **echo \$?** was executed. Since the output was **0**, it confirms that the compilation finished successfully without errors.

To ensure a bootable kernel image was generated, the commands **ls arch/x86/boot/bzImage** and **ls -la arch/x86/boot/bzImage** were run. The **ls** command checks if the file exists, while **ls -la** shows detailed information such as file size, permissions, ownership, and timestamp.

The presence of **bzImage** confirms that the kernel was successfully compiled and a bootable image is ready.

A terminal window titled 'sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17' showing the final steps of kernel compilation verification. It lists several kernel modules being built (BTF and LD), followed by three commands: 'echo \$?', 'ls arch/x86/boot/bzimage', and 'ls -la arch/x86/boot/bzImage'. The output shows the compilation was successful (0), the file 'bzImage' exists, and its details are listed.

```
sanamunir@sanamunir-VirtualBox: ~/kernel/linux-6.17
BTF [M] net/vmw_vsock/vmw_vsock_virtio_transport.ko
LD [M] net/vmw_vsock/vmw_vsock_virtio_transport_common.ko
LD [M] net/vmw_vsock/hv_sock.ko
BTF [M] net/vmw_vsock/vmw_vsock_virtio_transport_common.ko
BTF [M] net/vmw_vsock/hv_sock.ko
LD [M] net/vmw_vsock/vsock_loopback.ko
BTF [M] net/vmw_vsock/vsock_loopback.ko
LD [M] net/nsh/nsh.ko
LD [M] net/hsr/hsr.ko
BTF [M] net/nsh/nsh.ko
BTF [M] net/hsr/hsr.ko
LD [M] net/qrtr/qrtr.ko
BTF [M] net/qrtr/qrtr.ko
LD [M] net/qrtr/qrtr-smd.ko
BTF [M] net/qrtr/qrtr-smd.ko
LD [M] net/qrtr/qrtr-tun.ko
LD [M] net/qrtr/qrtr-mhi.ko
BTF [M] net/qrtr/qrtr-tun.ko
BTF [M] net/qrtr/qrtr-mhi.ko
LD [M] virt/lib/irqbypass.ko
BTF [M] virt/lib/irqbypass.ko
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ echo $?
0
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ ls arch/x86/boot/bzimage
ls: cannot access 'arch/x86/boot/bzimage': No such file or directory
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ echo $?
2
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ ls arch/x86/boot/bzImage
arch/x86/boot/bzImage
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ ls -la arch/x86/boot/bzImage
-rw-rw-r-- 1 sanamunir sanamunir 15954432 Feb 27 02:11 arch/x86/boot/bzImage
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 18: INSTALLING KERNEL MODULES

[Took 10 -15min]

The command `sudo make modules_install` successfully installed all compiled kernel modules into the system directory `/lib/modules/`. This prepares the system for the final kernel installation step using `make install`, after which the system can be rebooted into the newly compiled kernel.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ sudo make modules_install
[sudo] password for sanamunir:
SYMLINK /lib/modules/6.17.0-24K0573/build
INSTALL /lib/modules/6.17.0-24K0573/modules.order
INSTALL /lib/modules/6.17.0-24K0573/modules.builtin
INSTALL /lib/modules/6.17.0-24K0573/modules.builtin.modinfo
INSTALL /lib/modules/6.17.0-24K0573/modules.builtin.ranges
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/events/amd/amd-uncore.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/events/amd/amd-uncore.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/events/intel/intel-cstate.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/events/intel/intel-cstate.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/events/rapl.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/events/rapl.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/kernel/cpu/mce/mce-inject.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/kernel/cpu/mce/mce-inject.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/kernel/msr.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/kernel/msr.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/kernel/cpuid.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/kernel/cpuid.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/twofish-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/twofish-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/twofish-x86_64-3way.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/twofish-x86_64-3way.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/twofish-avx-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/twofish-avx-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/serpent-sse2-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/serpent-sse2-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/serpent-avx-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/serpent-avx2.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/des3_edc-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/des3_edc-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/camellia-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/camellia-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/camellia-aesni-avx-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/camellia-aesni-avx-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/camellia-aesni-avx2.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/camellia-aesni-avx2.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/blowfish-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/blowfish-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/cast5-avx-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/cast5-avx-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/cast6-avx-x86_64.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/cast6-avx-x86_64.ko
INSTALL /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/aegis128-aesni.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/arch/x86/crypto/aegis128-aesni.ko
```

Step 19: INSTALL KERNEL

The compiled kernel image and related files were installed into the /boot directory.

```
INSTALL /lib/modules/6.17.0-24K0573/kernel/virt/lib/irqbypass.ko
SIGN /lib/modules/6.17.0-24K0573/kernel/virt/lib/irqbypass.ko
DEPMOD /lib/modules/6.17.0-24K0573
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ sudo make install
[sudo] password for sanamunir:
INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/dkms 6.17.0-24K0573 /boot/vmlinuz-6.17.0-24K0573
* dkms: running auto installation service for kernel 6.17.0-24K0573
* dkms: autoinstall for kernel 6.17.0-24K0573 [ OK ]
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.17.0-24K0573 /boot/vmlinuz-6.17.0-24K0573
update-initramfs: Generating /boot/initrd.img-6.17.0-24K0573
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 6.17.0-24K0573 /boot/vmlinuz-6.17.0-24K0573
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.17.0-24K0573 /boot/vmlinuz-6.17.0-24K0573
run-parts: executing /etc/kernel/postinst.d/vboxadd 6.17.0-24K0573 /boot/vmlinuz-6.17.0-24K0573
VirtualBox Guest Additions: Building the modules for kernel 6.17.0-24K0573.

VirtualBox Guest Additions: Look at /var/log/vboxadd-setup.log to find out what
went wrong
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 6.17.0-24K0573 /boot/vmlinuz-6.17.0-24K0573
I: /boot/initrd.img.old is now a symlink to initrd.img-6.17.0-14-generic
I: /boot/initrd.img is now a symlink to initrd.img-6.17.0-24K0573
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.17.0-24K0573 /boot/vmlinuz-6.17.0-24K0573
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-24K0573
Found initrd image: /boot/initrd.img-6.17.0-24K0573
Found linux image: /boot/vmlinuz-6.17.0-14-generic
Found initrd image: /boot/initrd.img-6.17.0-14-generic
Found linux image: /boot/vmlinuz-6.14.0-37-generic
Found initrd image: /boot/initrd.img-6.14.0-37-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 20: CREATE A NEW INITIAL RAM FILESYSTEM (initramfs) FOR THIS KERNEL

To manually create a new **initial RAM filesystem (initramfs)** image for the newly installed custom Linux kernel version 6.17.0-24K0573.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ sudo update-initramfs -c -k 6.17.0-24K0573
update-initramfs: Generating /boot/initrd.img-6.17.0-24K0573
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 21: UPDATE GRUB BOOTLOADER CONFIGURATION FILE

To update the **GRUB bootloader configuration file** so that the newly installed kernel (6.17.0-24K0573) appears in the system boot menu.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ sudo update-grub
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-24K0573
Found initrd image: /boot/initrd.img-6.17.0-24K0573
Found linux image: /boot/vmlinuz-6.17.0-14-generic
Found initrd image: /boot/initrd.img-6.17.0-14-generic
Found linux image: /boot/vmlinuz-6.14.0-37-generic
Found initrd image: /boot/initrd.img-6.14.0-37-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 22: VERIFYING EXISTENCE OF THE initramfs IMAGE

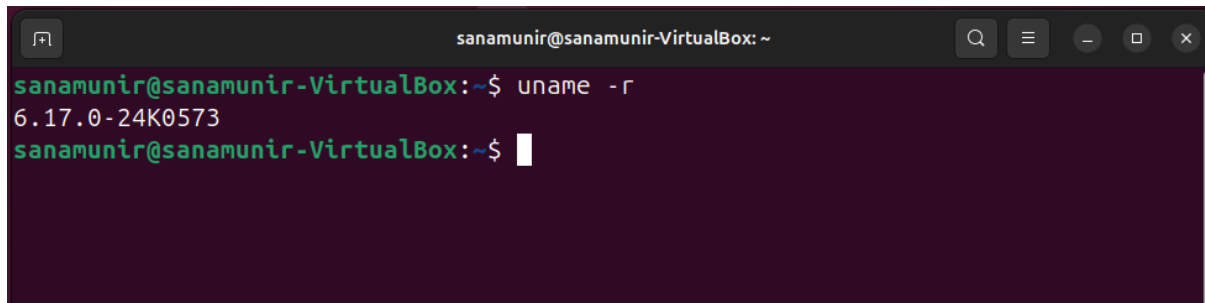
To verify that the newly installed custom Linux kernel version 6.17.0-24K0573 and its related files were successfully created in the /boot directory.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ ls -lh /boot | grep 6.17.0-24K0573
-rw-r--r-- 1 root root 295K Feb 27 05:06 config-6.17.0-24K0573
lrwxrwxrwx 1 root root 25 Feb 27 05:06 initrd.img -> initrd.img-6.17.0-24K0573
-rw-r--r-- 1 root root 581M Feb 27 05:12 initrd.img-6.17.0-24K0573
-rw-r--r-- 1 root root 9.2M Feb 27 05:06 System.map-6.17.0-24K0573
lrwxrwxrwx 1 root root 22 Feb 27 05:06 vmlinuz -> vmlinuz-6.17.0-24K0573
-rw-r--r-- 1 root root 16M Feb 27 05:06 vmlinuz-6.17.0-24K0573
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$
```

Step 21: **CUSTOM-BUILT KERNEL IS RUNNING.**

After installing and renaming the custom kernel, the command **sudo reboot** was executed. This restarts the system to apply the new kernel. Upon reboot, the system loads the newly installed kernel, making it active. This step ensures that all changes from the custom kernel are in effect.

```
sanamunir@sanamunir-VirtualBox:~/kernel/linux-6.17$ reboot
```

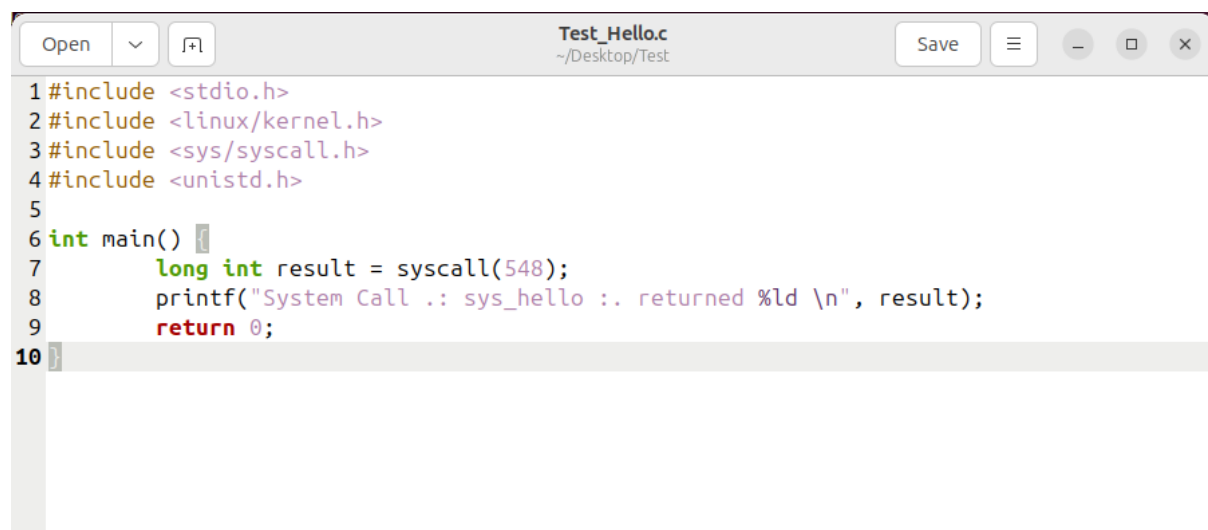
A terminal window titled "sanamunir@sanamunir-VirtualBox: ~" with standard window controls. The prompt is "sanamunir@sanamunir-VirtualBox:~\$". The command "uname -r" has been entered and executed, resulting in the output "6.17.0-24K0573". The prompt is now "sanamunir@sanamunir-VirtualBox:~\$" with a cursor.

```
sanamunir@sanamunir-VirtualBox:~$ uname -r
6.17.0-24K0573
sanamunir@sanamunir-VirtualBox:~$
```

Step 22: **SETTING UP THE WORKSPACE AND CREATING THE C PROGRAM**

A dedicated workspace named Test is created on the Desktop to organize the project files. The terminal navigates to this folder to prepare for coding. Using gedit, a new C program file Test_Hello.c is created, which will contain the code for the custom system call. This sets up the environment for writing and later compiling the program.

```
sanamunir@sanamunir-VirtualBox:~$ uname -r
6.17.0-24K0573
sanamunir@sanamunir-VirtualBox:~$ cd Desktop
sanamunir@sanamunir-VirtualBox:~/Desktop$ mkdir Test
sanamunir@sanamunir-VirtualBox:~/Desktop$ cd Test
sanamunir@sanamunir-VirtualBox:~/Desktop/Test$ gedit Test_Hello.c
```



```
Test_Hello.c
~/Desktop/Test

1 #include <stdio.h>
2 #include <linux/kernel.h>
3 #include <sys/syscall.h>
4 #include <unistd.h>
5
6 int main()
7     long int result = syscall(548);
8     printf("System Call .: sys_hello .: returned %ld \n", result);
9     return 0;
10
```

Step 23: **COMPILING AND EXECUTING THE CUSTOM SYSTEM CALL**

In this step, the C program Test_Hello.c was compiled using gcc, producing the executable a.out. Running ./a.out invoked the custom system call sys_hello, which executed in the kernel and returned 0 to indicate success. The output of the system call was printed to the kernel log using printk(). Finally, sudo dmesg was used to view the kernel messages, including the "Hello World" message from the custom system call.

```
sanamunir@sanamunir-VirtualBox:~/Desktop/Test$ gcc Test_Hello.c
sanamunir@sanamunir-VirtualBox:~/Desktop/Test$ ls
a.out  Test_Hello.c
sanamunir@sanamunir-VirtualBox:~/Desktop/Test$ ./a.out
System Call .: sys_hello :. returned 0
sanamunir@sanamunir-VirtualBox:~/Desktop/Test$ sudo dmesg
[sudo] password for sanamunir:
[ 0.000000] Linux version 6.17.0-24K0573 (sanamunir@sanamunir-VirtualBox) (gc
c (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42
) #1 SMP PREEMPT_DYNAMIC Fri Feb 27 02:04:51 PKT 2026
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-6.17.0-24K0573 root=UUID=d
befbf67-9a85-4594-923b-bd7f96556de8 ro quiet splash
[ 0.000000] KERNEL supported cpus:
[ 0.000000] Intel GenuineIntel
[ 0.000000] AMD AuthenticAMD
[ 0.000000] Hygon HygonGenuine
[ 0.000000] Centaur CentaurHauls
[ 0.000000] zhaoxin Shanghai
```

MANY MORE LINES IN BETWEEN (the command dumped all logs since reboot)

```
[ 594.448739] workqueue: e1000_watchdog [e1000] hogged CPU for >10000us 35 times, consider switching to WQ_UNBOUND
[ 615.586218] hrtimer: interrupt took 6922818 ns
[ 802.397320] workqueue: ata_sff_pio_task hogged CPU for >10000us 7 times, consider switching to WQ_UNBOUND
[ 917.061678] Hello World from my System Call!
```

TASK COMPLETED SUCCESSFULLY